

Introduction

For my project, I chose to try to speed up a script I am using for processing simulated fission data from a fission event generator called “FREYA”. Speeding up the process will save time in the long run and facilitate more coherent research, without having to wait for the results.

FREYA

FREYA stands for “Fast Reaction Event Yield Algorithm”. It allows the user to tune the fission process using 5 parameters:

- e0: Scales the level densities of the fission fragments
- x: Shifts excitation energy to light fragment as observed in experiments.
- c: Scales statistical fluctuations in the excitation energy of the fragments.
- cS: Scales the angular momentum distribution of the fission fragments.
- dTKE: Adjusts the total kinetic energy of the fragments to reproduce neutron data.

Using a mix of tables for experimental data, sampling and theoretical assumptions, FREYA then calculates fission fragments for many events which are produced in a file as in Figure 1.

Index	Z	A	E*	Ekin	TKE	Spin	#n	#g	Energies for neutrons and then gammas									
1	40	102	20.999	97.077	170.971	5.477	2	7	4.820	1.101	4.356	0.573	0.783	0.376	0.758	0.365	0.221	
1	52	134	11.326	73.894	170.971	7.630	1	4	0.983	0.858	0.472	0.760	1.143					
2	39	100	10.547	101.234	175.671	10.325	0	10	3.752	1.521	1.108	1.014	0.464	0.864	0.309	0.747	0.354	0.253
2	53	136	9.554	74.437	175.671	5.696	1	2	1.460	0.724	1.560							
3	38	97	15.094	96.111	163.181	2.691	2	3	0.653	0.192	1.088	0.909	0.675					
3	54	139	15.137	67.070	163.181	7.941	2	4	1.152	1.387	1.549	1.513	0.375	0.610				
4	41	104	16.753	100.602	179.864	4.608	2	4	2.669	1.214	2.195	0.326	0.515	0.201				
4	51	132	4.256	79.262	179.864	8.207	0	6	1.733	0.672	0.504	0.251	0.466	0.277				
5	36	93	11.381	99.319	163.911	5.714	1	5	0.575	3.091	1.339	0.746	1.093	1.514				
5	56	143	11.644	64.592	163.911	7.387	1	3	1.018	2.579	2.387	1.292						
6	35	87	15.577	93.482	148.065	3.116	2	1	3.129	0.392	0.579							
6	57	149	19.989	54.583	148.065	12.255	2	12	1.157	0.793	1.996	1.093	1.294	0.860	0.216	0.228	0.567	0.253
7	39	102	18.618	95.013	167.336	3.199	2	4	2.226	3.941	2.193	0.827	0.440	0.603				
7	53	134	8.998	72.323	167.336	1.318	1	2	0.952	1.298	0.916							
8	37	94	13.668	99.100	164.702	4.131	2	4	3.777	0.915	0.546	0.474	0.649	0.645				
8	55	142	9.683	65.602	164.702	1.193	1	5	1.472	2.402	1.420	0.414	0.249	0.395				
9	41	104	10.694	101.450	181.379	0.511	1	5	1.563	2.704	1.731	0.197	0.168	0.252				
9	51	132	8.800	79.930	181.379	6.162	1	1	4.102	1.225								
10	34	87	8.678	100.114	158.569	3.279	1	5	1.247	1.813	1.224	0.310	0.382	0.206				
10	58	149	15.009	58.456	158.569	4.308	2	4	3.287	1.886	0.441	0.711	0.256	0.281				
11	36	93	9.198	98.578	162.688	3.103	1	4	1.388	1.666	1.729	0.753	1.462					
11	56	143	15.049	64.110	162.688	8.345	2	3	5.685	0.128	0.187	0.715	0.586					
12	38	98	24.738	92.600	158.359	1.720	3	3	3.516	3.944	1.493	2.675	1.111	0.691				

Figure 1: Excerpt from a FREYA event-by-event file.

From this data so-called fission observables can be produced. Fission observables are data that can be experimentally observed and hence is most suitable for describing the nuclei and equally important for the verification of theory. The extraction of fission observable data from FREYA calculations is good for benchmarking the computer code. If it produces good results, then it can be used to make predictions and create nuclear data for the industry.

The downside is, that even if FREYA is not a pure Monte-Carlo code, it is still a rather time consuming process to produce the fission observable data and I would like to speed it up.

Optimization

For the attempt at optimization, I chose to try to “cythonize” the script. After a few feeble attempts I remembered a story a friend once told me. He wanted to start his own company and wanted to save

money by doing his own taxes. So he decided to get a Bachelor in Economics. After two years he realized that if he was going to do his own taxes, there would be no time left over for his work, and so he dropped out, started his company and hired an accountant. I realized the same thing after fiddling with Cython for a while. I now know that I know nothing. And since we discussed vibe coding in the lectures, I enlisted the help of ChaptGPT for the optimization of my script. I had the basics and could cythonize my script (with little gains in speed of execution), but the AI added syntax and swapped out the parts that would not benefit from cythonization. I am normally hesitant of using AI, but after reading the code I noticed that it is a faster way of learning by analyzing a finished product and comparing it to my entry-level-Python script.

Method

I created a copy of my script with the suffix pyx (as in the lectures) and wrote a setup file to compile my Python-script.

```
#!/usr/bin/env python3
from setuptools import setup
from Cython.Build import cythonize
setup(ext_modules = cythonize("Project_cython_produce_FREYA_Standalone_data.pyx"))
```

This version was later improved upon by AI to render:

```
#!/usr/bin/env python3
#from setuptools import setup
#from Cython.Build import cythonize
#setup(ext_modules = cythonize("Project_cython_produce_FREYA_Standalone_data.pyx"))

from setuptools import setup, Extension
from Cython.Build import cythonize
import numpy as np

# Define the extension module
extensions = [
    Extension(
        name="freya_fast",                # name of the compiled module (import this!)
        sources=["freya_fast.pyx"],        # your .pyx file
        include_dirs=[np.get_include()],    # required for NumPy
        extra_compile_args=["-O3"],         # optimization flag (important for speed)
    )
]

# Run setup
setup(
    name="freya_fast",
    ext_modules=cythonize(
        extensions,
        compiler_directives={
            "language_level": "3",          # use Python 3 syntax
            "boundscheck": False,           # disable bounds checking (faster)
            "wraparound": False,           # disable negative indexing (faster)
            "cdivision": True,              # faster C division
        },
    ),
)
```

As I mentioned before. This actually is educational. Even though I understand all the code, I would never have learnt all this in a week. But now I know how it should look.

The actual timing was done with a simple script:

```
#----- Timed run -----#
start = time.perf_counter()
result = cython_produce_FREYA_standalone_data(FREYA_data_path=data_path)
end = time.perf_counter()

print(f"Elapsed time: {end - start:.6f} seconds")
#-----#
```

My original "Project_default_produce_FREYA_Standalone_data.py"-script as well as the AI-improved version can be found in appendix A and B, respectively. (The files are also appended to the folder).

Results

I used two data sets for the trials. One smaller data set with 2 million lines of data, consisting of 1 million fission events with one line for the light and the heavy fission fragment for each event. This smaller set was used as a test set for working out the kinks in the program. The actual test was done with a larger data set with 10 million fission events (20 million lines of data).

The results for running both the default and the optimized scripts were:

Script	Execution time Default	Execution time Cython
10 ⁶ Fission events	10.2640 s	8.6866 seconds
10 ⁷ Fission events	67.9062 s	46.3176 seconds

As expected, the time difference for the smaller data set is not very large, but for the larger data set the Cython script executes 32% faster.

Conclusions and outlook

I honestly thought the gains would be greater. The chosen script might not be one where very large gains are to be made, but it has peaked my interest. The next step would be to investigate the possibility of multiprocessing. With my newly acquired knowledge, I know that the design layout of the basic script will affect the possibility of optimizing it, so I will keep that in mind. The previous gains in computational time I have made when going from Pandas to numpy were large and I am eager to tackle to multiprocessing in order to cut computational time further.

Appendix A

```
#!/usr/bin/python3
import numpy as np
import sys
import time
np.set_printoptions(threshold=sys.maxsize,linewidth=300)

if __name__ == "__main__":
    def default_produce_FREYA_standalone_data(FREYA_data_path):
        #=====
        #                                LOAD FREYA DATA
        #=====
        # [0]=2, [1]=A, [2]=E*, [3]=EKin, [4]=TKE, [5]=Rn, [6]=Rg
        ff_data = np.loadtxt(fname=FREYA_data_path,comments='#',usecols=(1,2,3,4,5,7,8))
        # BECAUSE EACH FRAGMENT HAS A ROW OF ITS OWN (2 ROWS/FISSION EVENT)
        num_fission_events = len(ff_data[:,0])/2
        unique_A, counts_unique_A = np.unique(ff_data[:,1],return_counts=True)
        yield_vec = np.zeros((len(unique_A),2))
        yield_vec[:,0] = unique_A
        yield_vec[:,1] = np.divide(counts_unique_A,num_fission_events)

        # NUBAR AND GUBAR
        nubar = np.sum(ff_data[:,5],axis=0)/num_fission_events
        gubar = np.sum(ff_data[:,6],axis=0)/num_fission_events
        print(f'Freya nubar = {nubar}, FREYA gubar = {gubar}')

        # NUBAR VS A
        An_vec = ff_data[:,1:5]
        N_vs_A_vec = np.zeros((len(unique_A),2))
        N_vs_A_vec[:,0] = unique_A
        for i,A in enumerate(unique_A):
            ind = np.where(An_vec[:,0] == A)[0]
            N_vs_A_vec[i,1] = np.mean(An_vec[ind,1])
        avg_nubar = np.average(N_vs_A_vec[:,1],weights=yield_vec[:,1])
        print(f'avg_nubar from N_vs_A: {avg_nubar}')

        # GUBAR VS A
        Ag_vec = ff_data[:,1:6]
        G_vs_A_vec = np.zeros((len(unique_A),2))
        G_vs_A_vec[:,0] = unique_A
        for i,A in enumerate(unique_A):
            ind = np.where(Ag_vec[:,0] == A)[0]
            G_vs_A_vec[i,1] = np.mean(Ag_vec[ind,1])
        avg_gubar = np.average(G_vs_A_vec[:,1],weights=yield_vec[:,1])
        print(f'avg_gubar from G_vs_A: {avg_gubar}')

        # PFNS & PFGS
        # CREATE ARRAYS FOR En AND Eg
        max_n = np.int16(np.max(ff_data[:,5]))
        max_g = np.int16(np.max(ff_data[:,6]))
        En_vec = np.zeros((len(ff_data[:,0]),max_n))
        Eg_vec = np.zeros((len(ff_data[:,0]),max_g))
        # READ AND TRUNCATE DATA
        with open(fname=FREYA_data_path,mode='r') as f:
            event_data = f.readlines()[24:]
            n_g_data = [line.strip().split()[7:] for line in event_data]
            for i,line in enumerate(n_g_data):
                num_n = int(line[0])
                num_g = int(line[1])
                En_vec[i,0:num_n] = line[2:2+num_n]
                Eg_vec[i,0:num_g] = line[2+num_n:2+num_n+num_g]
        # PREPARE FOR PLOTTING
        En_vec = En_vec.flatten()
        En_vec = En_vec[np.nonzero(En_vec)]
        avg_neutron_E = np.round(np.sum(En_vec)/(num_fission_events*nuubar),3)

        Eg_vec = Eg_vec.flatten()
        Eg_vec = Eg_vec[np.nonzero(Eg_vec)]
        avg_gamma_E = np.round(np.sum(Eg_vec)/(num_fission_events*gubar),3)

        # PRINT avg(En) AND avg(Eg)
        print(f'Freya avg(En) = {avg_neutron_E}, Freya avg(Eg) = {avg_gamma_E}')

        result_dict = {'yield_data':(yield_vec), 'nubar_data':(N_vs_A_vec,nuubar),
                        'Gubar_data':(G_vs_A_vec,gubar), 'En_data':(En_vec,avg_neutron_E),
                        'Eg_data':(Eg_vec,avg_gamma_E)}
        return result_dict

    #===== Opt Investigation =====#
    data_path = "...pathTo/Project../Project_APMP.out"
    #data_path = "...pathTo../263_FREYA_event_by_event_file/263_FREYALLNL_Tuckey2ndSetOptParamI236E253_1M.out"

    #===== DEFAULT =====#

    start = time.perf_counter()
    default_produce_FREYA_standalone_data(FREYA_data_path=data_path)
    end = time.perf_counter()
    print(f'Elapsed: {end - start:.4f} seconds')

    #===== RESULTS DEFAULT =====#
    # Default 1st script: Elapsed: 10.2640 seconds
    # Default 2nd script: Elapsed: 43.0064 seconds
```

Appendix B

```
#!/usr/bin/env python3
# cython: boundscheck=False, wraparound=False, cdivision=True
import numpy as np
cimport numpy as cnp
def cython_produce_FREYA_standalone_data(str FREYA_data_path):
    cdef:
        cnp.ndarray[double, ndim=2] ff_data
        Py_ssize_t n_rows
        int num_fission_events
    # LOAD DATA
    ff_data = np.loadtxt(fname=FREYA_data_path,
                        comments='#',
                        usecols=(1,2,3,4,5,7,8))
    n_rows = ff_data.shape[0]
    num_fission_events = n_rows // 2
    # =====
    # UNIQUE A + YIELDS
    # =====
    unique_A, counts_unique_A = np.unique(ff_data[:,1], return_counts=True)
    yield_vec = np.zeros((len(unique_A), 2), dtype=np.float64)
    yield_vec[:,0] = unique_A
    yield_vec[:,1] = counts_unique_A / num_fission_events
    # =====
    # NUBAR / GUBAR (Fast)
    # =====
    cdef double nubar = np.sum(ff_data[:,5]) / num_fission_events
    cdef double gubar = np.sum(ff_data[:,6]) / num_fission_events
    print(f'FREYA nubar = {nubar}, FREYA gubar = {gubar}')
    # =====
    # NUBAR vs A (optimized loop)
    # =====
    cdef:
        Py_ssize_t i, j, nA = len(unique_A)
        double A
        double[:, :] data_view = ff_data
        double[:, :] N_vs_A_vec = np.zeros((nA, 2), dtype=np.float64)
        double sum_val
        int count
    for i in range(nA):
        A = unique_A[i]
        N_vs_A_vec[i,0] = A
        sum_val = 0.0
        count = 0
        for j in range(n_rows):
            if data_view[j,1] == A:
                sum_val += data_view[j,5]
                count += 1
        if count > 0:
            N_vs_A_vec[i,1] = sum_val / count
    avg_nubar = np.average(N_vs_A_vec[:,1], weights=yield_vec[:,1])
    print(f'avg_nubar from N_vs_A: {avg_nubar}')
    # =====
    # GUBAR vs A (same pattern)
    # =====
    cdef double[:, :] G_vs_A_vec = np.zeros((nA, 2), dtype=np.float64)
    for i in range(nA):
        A = unique_A[i]
        G_vs_A_vec[i,0] = A
        sum_val = 0.0
        count = 0
        for j in range(n_rows):
            if data_view[j,1] == A:
                sum_val += data_view[j,6]
                count += 1
        if count > 0:
            G_vs_A_vec[i,1] = sum_val / count
    avg_gubar = np.average(G_vs_A_vec[:,1], weights=yield_vec[:,1])
    print(f'avg_gubar from G_vs_A: {avg_gubar}')
    # =====
    # ENERGY PART (still Python-heavy)
    # =====
    max_n = int(np.max(ff_data[:,5]))
    max_g = int(np.max(ff_data[:,6]))
    En_vec = np.zeros((n_rows, max_n))
    Eg_vec = np.zeros((n_rows, max_g))
    with open(FREYA_data_path, 'r') as f:
        event_data = f.readlines()[24:]
    cdef list parts
    cdef list num_n, num_g
    for i in range(len(event_data)):
        parts = event_data[i].split()[7:]
        num_n = int(parts[0])
        num_g = int(parts[1])
        En_vec[i, 0:num_n] = parts[2:2+num_n]
        Eg_vec[i, 0:num_g] = parts[2+num_n:2+num_n+num_g]
    En_vec = En_vec.ravel()
    En_vec = En_vec[En_vec != 0]
    Eg_vec = Eg_vec.ravel()
    Eg_vec = Eg_vec[Eg_vec != 0]
    avg_neutron_E = np.sum(En_vec) / (num_fission_events * nubar)
    avg_gamma_E = np.sum(Eg_vec) / (num_fission_events * gubar)
    print(f'FREYA avg(En) = {avg_neutron_E}, FREYA avg(Eg) = {avg_gamma_E}')
    return {
        'yield_data': (yield_vec,),
        'Nubar_data': (np.asarray(N_vs_A_vec), nubar),
        'Gubar_data': (np.asarray(G_vs_A_vec), gubar),
        'En_data': (En_vec, avg_neutron_E),
        'Eg_data': (Eg_vec, avg_gamma_E)
```